

Relatório técnico Artist Alley

Leonardo de Souza Galvão

Introdução

Artist Alley é uma plataforma móvel para gerenciamento de commissions artísticas. O sistema permite que artistas digitais publiquem vagas de encomenda de obras e que clientes realizem pedidos diretamente pelo aplicativo, acompanhando o status de suas encomendas em tempo real.

A solução é composta por dois módulos principais: um backend em Node.js com Express e TypeScript, e um aplicativo móvel em Flutter. A comunicação assíncrona entre os módulos é mediada pelo RabbitMQ, enquanto a entrega de notificações ao cliente móvel ocorre por meio de Server-Sent Events (SSE).

Arquitetura Implementada

O fluxo funciona da seguinte forma: o aplicativo Flutter realiza requisições REST ao backend, que persiste os dados no PostgreSQL via Prisma ORM e publica eventos no RabbitMQ. O consumer, rodando no mesmo processo do servidor, consome esses eventos e notifica os clientes conectados por meio de SSE, que são canais HTTP persistentes unidirecionais do servidor para o aplicativo.

Mensageria e eventos

Dois exchanges do tipo topic são declarados no RabbitMQ: `orders_exchange`, que recebe eventos com routing key `order.`, e `commission_exchange`, com routing key `commission.`. As filas `orders_queue` e `commission_queue` são duráveis e ligadas aos respectivos exchanges. Os eventos publicados são:

- `ORDER_CREATED`: disparado quando um cliente cria um pedido.
- `ORDER_STATUS_CHANGED`: disparado quando o artista altera o status de um pedido via `PUT /orders/:id`, bem como ao aceitar (`ACCEPTED`) ou rejeitar (`CANCELLED`) via rotas dedicadas.
- `COMMISSION_ORDER_QUEUED`: disparado junto ao `ORDER_CREATED`, informando a posição do pedido na fila da `commission`.

Camadas do Backend

O backend foi organizado segundo os princípios da Clean Architecture, separando claramente as responsabilidades:

- Rotas (src/routes/): definem os endpoints REST e delegam para os serviços. Respondem apenas por validação de entrada e serialização de saída.
- Serviços (src/services/): contêm a lógica de negócio. OrderService, CommissionSlotService e UserService são os três pilares domânicos.
- EventService: responsável exclusivamente por publicar eventos no RabbitMQ, separando a lógica de mensageria da lógica de domínio.
- Lib (src/lib/): encapsula a infraestrutura de conexão — rabbitmq.ts para o broker e prisma.ts para o ORM.
- Middleware (src/middleware/): auth.ts implementa autenticação JWT; sse.ts gerencia os canais SSE e a função notifyArtist().
- Types (src/types/): define os contratos tipados dos eventos de domínio (OrderCreatedEvent, OrderStatusChangedEvent, etc.).

Camada de tempo Real (SSE)

O middleware sse.ts mantém um Map<string, Response[]> indexado pelo userId do artista. Quando o consumer processa um evento, chama notifyArtist(artistId, payload), que itera sobre todas as conexões SSE ativas daquele artista e envia o payload serializado em JSON. O aplicativo Flutter mantém a conexão aberta indefinidamente via EventService (singleton), reconectando automaticamente após falhas com delay de 5 segundos.

SSE em vez de WebSocket

A escolha de Server-Sent Events (SSE) para a camada de tempo real foi deliberada. O canal de notificação é estritamente unidirecional — o servidor informa o cliente, e o cliente responde por REST —, o que elimina a necessidade da bidirecionalidade do WebSocket. SSE é um protocolo HTTP padrão, não requer bibliotecas adicionais no servidor Express nem no Flutter (o pacote http nativo é suficiente), e reconecta nativamente em caso de queda. Essa simplicidade reduziu significativamente a superfície de erro em comparação com uma implementação WebSocket completa.

RabbitMQ e Topic Exchange

O uso de topic exchange permite granularidade no roteamento. As routing keys `order.*` e `commission.*` separam semanticamente os dois domínios de eventos, permitindo que futuros consumidores se inscrevam em subconjuntos específicos sem alterações na topologia. A persistência das filas (`durable: true`) e das mensagens (`persistent: true`) garante que eventos não sejam perdidos mesmo em caso de reinicialização do broker, atendendo ao requisito de confiabilidade da mensageria.

Separação entre EventService e OrderService

A lógica de publicação de eventos foi isolada em um EventService dedicado, seguindo o princípio da responsabilidade única (SRP). O OrderService conhece apenas as operações de domínio (criar, buscar, atualizar) e delega para o EventService a produção de mensagens. Isso facilita testes unitários do domínio e permite substituir o broker sem alterar a lógica de negócio.

Atualização da UI guiada pelo evento, não pela resposta HTTP

Uma decisão crítica de design foi fazer a interface do artista atualizar sua lista de pedidos unicamente em resposta a eventos SSE, e não à resposta HTTP das ações (aceitar, recusar, alterar status). Isso garante que o fluxo completo RabbitMQ → Consumer → SSE → Flutter seja exercitado em cada operação, comprovando o funcionamento do pipeline de eventos.

Dificuldades encontradas

O problema mais crítico encontrado foi que a rota SSE (`/events/stream/:userId`) não estava registrada no arquivo `app.ts`. O router era exportado corretamente por `sse.ts`, mas nunca era montado no app Express, fazendo com que todas as conexões SSE retornassem 404. O Flutter recebia o erro, aguardava 5 segundos e tentava reconectar indefinidamente, sem nunca estabelecer o canal. A solução foi adicionar `app.use('/events', sseRouter)` ao `app.ts`.

A função `findByArtistId` no OrderService usava `prisma.order.findFirst()`, retornando no máximo um pedido por artista. A rota `GET /orders/artist/:artistId` chamava essa função, fazendo com que artistas com múltiplos pedidos vissem apenas o mais antigo. A

solução foi criar `findAllByArtistId` com `findMany` e incluir de `client` e `commissionSlot`, e atualizar a rota para usar a nova função.

Reflexão sobre os Padrões Estudados

Event-Driven Architecture (EDA)

A Arquitetura Orientada a Eventos propõe que componentes do sistema se comuniquem por meio de eventos assíncronos em vez de chamadas diretas, promovendo desacoplamento temporal e espacial (FOWLER, 2017). No Artist Alley, esse princípio se manifesta na relação entre `OrderService` e o `consumer`: o primeiro não conhece nem depende do segundo. A criação de um pedido dispara um evento `ORDER_CREATED` no RabbitMQ, e qualquer componente interessado pode reagir a ele independentemente. Isso permite, por exemplo, adicionar futuramente um serviço de notificação push ou e-mail simplesmente consumindo a mesma fila, sem modificar o `OrderService`.

A experiência prática do projeto evidenciou tanto os benefícios quanto os desafios da EDA. O desacoplamento facilita a evolução independente dos componentes, mas exige disciplina na definição e documentação dos contratos de eventos. A ausência de tipos de evento bem definidos (como o `ORDER_UPDATE` inexistente) causou falha sutil e difícil de diagnosticar.

Message-Oriented Middleware (MOM)

O RabbitMQ atua como middleware orientado a mensagens, desempenhando o papel de broker entre produtores e consumidores (HOHPE; WOOLF, 2003). O padrão `Publisher-Subscriber`, implementado via `topic exchange`, permite que múltiplos consumidores se inscrevam em eventos com diferentes níveis de granularidade. A garantia de entrega é reforçada pelo uso de filas duráveis e mensagens persistentes, e a confirmação explícita (`ack/nack`) assegura que mensagens não sejam perdidas em caso de falha no processamento.

Referências

Stopford, Ben. *Designing event-driven systems*. O'Reilly Media, Incorporated, 2018.

Hohpe, Gregor, and Bobby Woolf. *Enterprise integration patterns: Designing, building, and deploying messaging solutions*. Addison-Wesley Professional, 2004.

Martin, Robert C. *Clean architecture: a craftsman's guide to software structure and design*. Prentice Hall Press, 2017.